Shortening and simplifying the DiophantineConsistency Lean proofs

Source specific proposals with auditable examples

ProveIt commit `60e9c5d620c6c387d932876cfa2d9376f64ea56c`

1 The best first changes

Start by deleting repeated reasoning, not by introducing a large tactic framework. The clearest opportunities are a common quotient/remainder certificate theorem, existing list lookup lemmas, shared scanner iteration facts, and local algebra proofs that can use narrower facts. The largest reusable mathematical component is the identical free-carrier polynomial tail in the Split and Gap developments. The biggest generated files need a different treatment: their named intermediate lemmas may be valuable performance boundaries.

Nine concrete algebra replacement bodies are supplied. Their combined nonblank proof lines fall from **123 to 56**, and their character count from **4,835 to 2,533**. These are **static textual measurements**, not compiled savings or performance results. All nine declaration headers are unchanged. Five algebra patches pass a textual applicability check. The proposals still require Lean elaboration and downstream validation.

Priority	Change	Main benefit and boundary
First	Reuse core list facts and the DivMod certificate	Remove duplicate inductions; keep both encodings.
First	Local positivity, units and sum-of-squares proofs	Same statements; shorter proof bodies.
Next	Shared iteration and exact checker specifications	Fewer scanner inductions; programs unchanged.
Next	Shared polynomial tail and guarded transport	One proof per common component; distinct recipes.
Pilot first	Native contexts and finite dispatch	Make large obligations local and staged.
Later	Generated DAG certificates and recipe generators	Reduce duplication without losing useful cutpoints.

What this report does not claim. It does not produce a smaller equation, alter a coefficient tuple, establish a new consistency theorem, or close the remaining whole native-machine/internal-ZFC links. Shorter Lean source does not imply a smaller proof term or faster checking. Each proposal identifies the contract it must preserve.

How to use the archive. Read the low-risk examples first, then the interface proposals. Lean candidates, patches, source inventories and measurement scripts make the suggestions reproducible. Treat candidate code as review material until the validation gates in Section 16 are met.

2 Scope and existing structure

The review is pinned to the commit printed on the first page, not a moving `main` branch. The repository pins Lean `v4.32.0` and mathlib `v4.32.0`. Source links point to that ProveIt commit. Unless stated otherwise, modules are relative to

`Computability/DiophantineConsistency/Lean/DiophantineConsistency/`

Static inventory of the main Lean directory	Count
Lean files	546
Physical source lines	73,971
Lexically detected theorem and lemma declarations	4,692
Files explicitly marked with Generated by	18
Lines in those marked files	14,807
Direct imports in the main umbrella	283

These are source-analysis metrics, not a Lean syntax-tree census. The broader evidence inventory covers 711 Lean files and 104,658 lines, including preserved research/provenance material outside the main Lean directory. Its denominator must not be mixed with the table above. A generator marker is only a lower-bound ownership signal: unmarked files can contain generated material.

Preserve the good interfaces already present.

- `ArithmeticDAGSemantics.lean:58–116` already supplies local node equations and converse reconstruction. Reuse this API rather than creating a second semantics library.
- `NativeFormulaGraphs.lean:22–26` already centralizes a correctness bridge in a small private `realize` helper. A repeated argument disappears without hiding the surrounding mathematics.
- The representation/register layers separate semantic predicates from witness counts. These cost contracts are part of the result, not incidental implementation details.
- The Split/Gap development already separates source bounds, certificates, polynomial formulas, DAG steps and recipes. Preserve these stages and their premises.
- Audit/check modules document dependencies and boundary cases. Their repetition is not, by itself, a reason to delete them.

The mathematical endpoint is not being strengthened. The pinned overview reports checked arithmetic and machine checkpoints while leaving the complete polynomial-to-native tile/machine connection and an internal-ZFC derivation open. It distinguishes an external Lean consistency theorem from an object-level ZFC proof. This report concerns proof engineering within that boundary, not equation search or logical completion. See `README.md:24–29` and `README.md:828–846`.

A useful refactoring preserves at least three layers: the mathematical statement, the executable encoding, and explicit quantitative theorems. The first can remain unchanged while the second or third changes accidentally. Subsequent sections therefore separate these obligations.

3 Small local proofs with less repetition

Let elaboration choose a coordinate already fixed by the goal. In Carrier66.lean:165–203, `drop_pos` and `restore_pos` split a finite coordinate and then enumerate almost every application of the same hypothesis. After `fin_cases`, the goal determines the argument to `hv`. The candidate bodies are:

```
-- drop_pos: same theorem statement
  intro j
  fin_cases j <;> first
    | exact hn
    | exact hv _

-- restore_pos: same theorem statement
  intro j
  fin_cases j <;> first
    | exact hh
    | exact ht
    | exact hv _
```

The first body falls from 16 to 4 nonblank lines; the second from 17 to 5. The proof now expresses the reason for positivity rather than a list of coordinate numerals. It remains explicit about the inserted witness and the special `tau` coordinate.

Why this is a good pilot. The finite index types and positivity hypotheses remain exactly as before. The patch changes neither `drop`, `restore`, witness reconstruction nor the polynomial. If an index map changes later, the proof still states the structural obligation. Lean must nevertheless verify that reduction exposes the expected coordinate; a plausible metavariable is not a proof receipt.

Use a narrow rewrite set when canonicality supplies every equation. In Carrier66.lean:158–163, `canonical_zero` extracts six equalities, unfolds the value, rewrites, and calls `ring`. After the same extraction, the proposed body ends with:

```
  simp only [value, hP, hQ, hU, hi, hW, hZ,
    one_mul, sub_self]
```

The benefit is narrower automation and a direct connection between hypothesis and goal. It is *not* a byte-saving example: the complete body has 2 rather than 4 nonblank lines but **106 rather than 101 characters**. Keep it only if readability or elaboration behavior improves.

Avoid indiscriminate compression. A single `aesop` or `simp_all` over many unrelated facts can make a short proof more context-sensitive. These examples work as proposals because the search space is small and intended facts are visible. Do not generalize this style to large generated polynomial goals without measurement. The archive preserves exact before/after text and unchanged headers, so the compiler can test the actual candidates rather than a paraphrase.

4 Algebraic facts instead of repeated arithmetic search

Sum of squares. ShortSystem.lean:63–88 proves that a five-term residual sum is zero exactly when each residual vanishes. Over the integers each square is nonnegative, so `add_eq_zero_iff_of_nonneg` decomposes the sum; `sq_eq_zero_iff` removes squares and `sub_eq_zero` removes subtraction. The candidate is:

```
simp (disch := positivity) only
  [residualSum, add_eq_zero_iff_of_nonneg,
   sq_eq_zero_iff, sub_eq_zero, and_assoc]
```

It replaces 23 nonblank proof lines with 2. Its mathematical content is

$$(n_0 - 1)^2 + (n_1 - 1)^2 + (n_a - 1)^2 + (n_s - 1)^2 + (n_k n_t - 1)^2 = 0$$

if and only if all five displayed residuals vanish. This uses the ordered-ring zero-sum fact directly instead of deriving each nonnegativity consequence by repeated arithmetic search.

Units of the integers. The private helper at ShortSystem.lean:20–24 can use a pinned library theorem:

```
exact (Int.eq_one_or_neg_one_of_mul_eq_one h).resolve_right hne
```

This changes 4 body lines to 1. The inspected API gives the required disjunction for an integer factor of a product equal to one. Its exclusion of -1 is essential; the corresponding natural-number argument has different premises.

Use that helper uniformly in the product theorem. In ShortSystem.lean:36–57, the candidate extracts `ns = 1` first, then uses the conditional assumption that `ns = 1` excludes `na = -1`. Each factor application normalizes only associativity and commutativity of multiplication. The remaining product reduces with the extracted equalities. This cuts 17 to 13 body lines while exposing the dependency order.

Risks and acceptance. The sum-of-squares simplifier must discharge all side conditions without using a stronger domain assumption. Check the exact conjunct order expected by clients. For the product proof, retain every sign exclusion: a product equal to one in $\mathbb{Z}$ does not make every factor positive. Keep signatures unchanged and inspect axiom reports after compilation. If the narrow simplifier becomes fragile, a small named nonnegative-sum lemma is preferable to a broader global simp set.

The selected reductions are meaningful because the repeated argument is standard algebra already represented in the library. They do not justify replacing every arithmetic proof by automation: proofs of positivity, Pell bounds or quotient restoration can depend on domain details that these simple lemmas do not capture.

5 Structural induction and domain transport

Combine genuinely identical constructor branches. In QuarticCompiler.lean:101–121, `Gates.complete` repeats the same proof for addition and multiplication. The candidate uses one alternative-pattern branch:

```
| add p q hp hq | mul p q hp hq =>
    intro k
    cases k with
    | inl j => simp [residualSyntax, Expr.eval,
        natAssignment, values, values_root]
    | inr k =>
        cases k with
        | inl k => simpa [residualSyntax,
            Expr.eval_rename, assignment_comp, values] using hp k
        | inr k => simpa [residualSyntax,
            Expr.eval_rename, assignment_comp, values] using hq k
```

The full body, including the induction setup and leaf cases, goes from 19 to 11 nonblank lines. The common reasoning is structural: the root residual vanishes by evaluation, and child residuals vanish by induction. Keep the constructor definitions separate; only their proof skeleton is shared.

Use the map lemma at the recursive boundary. In ArithmeticDAG.lean:66–77, `run_map` needs induction on the node list and a rewrite of the appended new value. Its successor case becomes:

```
  simpa only [run, List.map_append, List.map_singleton,
    Node.eval_map] using
      ih (values ++ [n.eval p x values])
```

This is a modest 7-to-5-line body reduction. It emphasizes the invariant: mapping computed values commutes with evaluating the next node. It does not suggest unfolding the whole run.

Cast the whole conclusion instead of splitting it manually. FirstPellCarrier.lean:58–77 already has a good domain boundary: it lifts positive integers to naturals, proves the norm equality and invokes `first_norm_facts`. Preserve that design. The actual proposal only replaces the final case split on `hgap` by `by exact_mod_cast hgap`, transporting the entire disjunction at once. This modest change reduces 16 body lines to 13.

Natural subtraction and integer subtraction are not interchangeable. Divisibility needs its own checked coercion path. The complete candidate retains these details rather than treating `exact_mod_cast` as an unrestricted solver. The recommended pattern isolates a domain boundary rather than hiding it.

Acceptance. Compile the exact replacement in its original namespace/import context, then its clients. An extracted example can miss local instances or reducibility settings. Check natural-versus-signed witness counts and quartic completeness statements remain unchanged; combining proof branches must not change the compiler's executable representation.

6 Two small shared APIs worth adopting first

Delete duplicated list lookup inductions. Pinned Lean core already supplies the relevant facts:

```
exact List.mem_of_getElem? h
exact List.mem_iff_getElem?.mp h
```

The first turns successful optional lookup into membership; the second turns membership into an index with successful lookup. The API was inspected in Lean 4.32 core list lemmas, lines 479–486. Compilation at these call sites remains a separate check.

Five private membership inductions occur at `AnnotatedRename.lean:110–118`, `AnnotatedBound.lean:50–58`, `AnnotatedProofChecker.lean:177–188`, `AnnotatedAssumption.lean:40–48`, and `AnnotatedRuleSide.lean:18–26`. Two existence inductions follow the latter pair: `AnnotatedAssumption.lean:50–57` and `AnnotatedRuleSide.lean:28–35`. Use qualified core names and remove the obsolete helpers. Do not create a new general list lemma: the general theorem exists.

The supplied `standard_library_cleanup.patch` also replaces the Option injectivity case split at `AnnotatedParents.lean:104–107` by `Option.map_injective judgmentCode_injective`. Across six files it removes seven duplicated helper declarations, adds 14 physical lines and removes 84, a net decrease of **70 lines**. The patch applies cleanly; it is not Lean-compiled. This file-level diff is a different metric from the nine algebra proof-body measurements.

Share the quotient/remainder certificate, not its realizers. The 17-line proofs at `QuadraticDivMod.lean:10–26` and `RegisterDivMod.lean:9–25` have the same statement and proof:

$$(\exists s \in \mathbb{N},\ n = qb + r \ \wedge\ b = r + s + 1) \quad \Longleftrightarrow \quad 0 < b \ \wedge\ q = n/b \ \wedge\ r = n \bmod b.$$

Move this arithmetic fact into a small neutral module and keep both public `certificate_iff` names as wrappers. `ProposedDivModCert.lean` copies the existing proof into that interface. A one-module alias is an easy pilot, but a neutral dependency is cleaner than making one encoding depend on the other.

The witnesses are intentionally different. The quadratic representation uses **1 witness**; the register representation uses **23**. See `QuadraticDivMod.lean:65–65` and `RegisterDivMod.lean:79–79`. Sharing the arithmetic does not identify these programs or optimize their costs. Keep representation definitions and cost theorems intact, ideally with their current `rfl` proofs.

Acceptance. Check all helper consumers before removal; compile affected modules and reverse dependencies; preserve signatures and axiom sets. Both cost checks must still reduce. Narrow the prototype's broad tactic import after the shared proof works. The production change should shrink a dependency boundary, not add a large umbrella to save a small proof.

7 Replace scanner inductions with an iteration bridge

Five scanners separately prove that zero stays zero, a completed state stays completed, and a successor iteration can move to the initial state. There are **15 such proofs with 72 nonblank body lines**. They can become one-term wrappers after a shared bridge. Locations: `AnnotatedRenameScan.lean:34–55`, `AnnotatedBoundScan.lean:30–51`, `AnnotatedLineScan.lean:35–54`, `AnnotatedContext.lean:100–119`, and `AnnotatedAxiomContext.lean:32–50`.

Keep executable `Nat.rec` definitions unchanged. Prove once that the existing recursion equals mathematical iteration:

$$\mathrm{Nat.rec}\ \ x\ (\lambda_\ a, f(a))\ n = f^{[n]}(x).$$

The preferred candidate, `IterationMathlib.lean`, reuses `Function.iterate_fixed` and `Function.Semiconj.iterate_right`. These APIs were inspected in the pinned mathlib revision. The core-only `IterationCore.lean` is an independent prototype, not a recommendation to maintain a second iteration library.

Representative wrappers from `ScannerReplacements.lean` are:

```
example (fuel : Nat) : scan fuel 0 = 0 :=
  rec_fixed step fuel 0 step_zero

example (fuel state : Nat) :
    scan (fuel + 1) state = scan fuel (step state) :=
  rec_succ_right step fuel state

example (fuel history : Nat) :
    scan fuel (Nat.pair 0 history) = Nat.pair 0 history :=
  rec_fixed step fuel _ (by simp [step])
```

The rename scanner closes over its mode and parameter. Two further uses appear in `AnnotatedContextLookup.lean:56–76`: its right-step identity and the fixed-point argument in the nil case.

Encoding fusion is a second reusable fact. If $g(e(x)) = e(f(x))$ for one transition, then $g^{[n]}(e(x)) = e(f^{[n]}(x))$. This can replace repeated coding-versus-execution inductions. The closure candidates target `AnnotatedClosure.lean:26–32` and `AnnotatedClosure.lean:126–132`. Their premise is commutation on *genuine formula codes*, not equality of numeric step functions on arbitrary naturals. For only two short inductions, helper overhead may outweigh the benefit; additional consumers make the interface more attractive.

Keep distinct invariants distinct. A line scanner's provability history differs from a rename scanner's valid rows. Soundness at arbitrary supplied fuel differs from completeness at sufficient fuel. Do not add parser-validity assumptions or remove absorbing/finished states. The 72-to-15 wrapper comparison excludes shared bridge and import costs. Its main value is one maintenance point for mechanics while leaving semantic invariants visible.

8 Exact checker outputs and proof side dispatch

A Boolean correctness theorem says when a numeric checker returns one. Clients needing the actual number must also reason about the zero case. Repeating that conversion obscures control flow.

For numeric output c and Boolean predicate b, use the interface

$$c \in \{0, 1\}, \qquad c = 1 \longleftrightarrow b = \text{true} \quad \Longrightarrow \quad c = \text{truth}(b).$$

The range premise is indispensable: an invalid input could return 2 and still satisfy the equivalence with a false Boolean. The helper splits the Boolean; the false case uses the range fact, while the true case uses completeness directly.

Two concrete consumers. `NumericCheckerReplacements.lean` derives exact `localCheck_eq_truth_candidate` specifications from existing Booleanity and correctness theorems. The `step_cons` proofs at `AnnotatedBoundScan.lean:53–65` and `AnnotatedRenameScan.lean:57–69` then split the source Boolean and simplify the encoded branch with one output equation. Keep `localCheck`, `step`, row coding and table coding unchanged.

Verify that the range theorem covers exactly the raw inputs required by a consumer. Also check the definitional relationship between `tableCode seen` and the corresponding `listCode` of mapped row codes. The candidate uses a local `change` to expose that relationship. A standalone helper plus local specifications is preferable to an oversized checker typeclass.

Normalize the common tag dispatcher at proof level. `AnnotatedBoundLocal.lean:55–79` and `AnnotatedRenameLocal.lean:63–87` repeat eight-tag case analysis and arithmetic exclusions. The proposed `threshold_dispatch_eq` theorem identifies the same five outcomes of the frozen tests: tags 0/1; tag 2; tags 3/4/5; tags 6/7; and invalid tags. The current executable tests use natural truncated subtraction, so the normalization proof must account for that rather than reading subtraction as integer subtraction.

The candidate theorem quantifies over *all natural tags*. Adding `tag < 8` would weaken raw-input soundness. `DispatchReplacements.lean` gives the common normalization and two target bodies. This is less mature than core list reuse: measure its `split_ifs`/`omega` elaboration before adopting it.

A later shared fold. The run/append/invariant proofs in `AnnotatedRename.lean:192–231`, `AnnotatedBound.lean:132–171`, and `AnnotatedProofChecker.lean:199–240` have a common guarded-history shape. A proof-side adapter to a guarded fold can share append and preservation lemmas, with identity output for rows and rule output for proof lines. First bridge existing recursive functions rather than changing them. This remains a design proposal, not a supplied checked refactor, and is justified only if multiple real consumers become clearer.

9 Share the identical polynomial tail

The 24-line `ratioValue` definitions in `SplitNinePolynomial59.lean:58–81` and `GapNinePolynomial66.lean:59–82` are text-identical. The larger recipes are not. This is the best medium-scale mathematical sharing opportunity because its boundary is already visible in source.

The common tail takes twelve scalar inputs $R, N, b, g, t, h, s, w, C, i, j, n$ and introduces 22 intermediate quantities, ending in `NearSquareFinalizer.norm`. A neutral `FreeCarrierPolynomial` module should own this construction and its map/cast theorems. `ProposedSharedTail.lean` is an uncompiled extraction prototype; compatibility examples ask Lean to prove both old scalar values equal to it by `rfl`.

Mapping laws belong with the common expression. The map proofs at `SplitNinePolynomial59.lean:308–312` and `GapNinePolynomial66.lean:301–305` repeat, as do natural specializations at `SplitNinePolynomial59.lean:341–345` and `GapNinePolynomial66.lean:334–338`. Prove them once over a commutative ring, then derive the larger recipes' laws compositionally.

Ingredient	Gap recipe	Split recipe
Scalar tail	Same `ratioValue`	Same `ratioValue`
Block length	$b\,\mathrm{base}^{2200}$	$b\,\mathrm{base}^{2279}$
Threshold	$\mathrm{scale}\,\mathrm{base}^{1162}$	$\mathrm{scale}\,b\,\mathrm{base}^{1201}$
Height and scale	Recipe-specific powers of 2	Different powers of 2

The exponents and the extra factor of b are not cosmetic. A configuration record may eventually express the recipe differences, but extracting the identical tail requires no such redesign. Keep source/threshold definitions separate in the first patch.

Preserve names and reduction behavior. Retain aliases or thin wrappers for existing clients and generated code. Test whether an abbreviation or definition gives the intended definitional equality before changing the API. Avoid unfolding 22 lets everywhere: the point is to retain sharing, not duplicate the expanded polynomial in many proofs.

Acceptance. Check scalar equality, polynomial-data equality, map laws, natural specializations and coefficient/recipe clients. Equality at finitely many sampled assignments is not polynomial equality. Keep the coefficient tuple, exponents, degree and operation counts fixed. Measure total maintained source including wrappers; extraction can reduce duplicated reasoning before it reduces total bytes.

A separate follow-on can share the generated proof that this tail matches its circuit registers. Do not combine those two refactors initially. The scalar extraction has a much smaller validation surface and can establish the abstraction boundary before performance-sensitive DAG clients migrate.

10 Guarded solution transport must preserve domains

Carrier developments move between witness presentations. Their common proof shape is an equivalence of *existence of solutions*, not necessarily a bijection of assignments. A proof API should reflect that weaker and more useful contract.

For a domain predicate D and equation predicate P, put

$$\mathrm{HasSolution}(D, P) := \exists x,\ D(x) \wedge P(x).$$

A transport from (D_A, P_A) to (D_B, P_B) supplies

$$D_A(x) \wedge P_A(x) \Longrightarrow \exists y,\ D_B(y) \wedge P_B(y),$$
$$D_B(y) \wedge P_B(y) \Longrightarrow \exists x,\ D_A(x) \wedge P_A(x).$$

It yields solvability equivalence and composes with a second transport. The core-only `SolutionTransport.lean` provides `exists_iff`, `trans`, and same-domain congruence under a guard.

Why not demand inverse functions? Witness maps can introduce auxiliary witnesses, choose fresh Pell seeds, forget redundant coordinates, or be many-to-one. Requiring an equivalence of witness types adds irrelevant inverse obligations and can be false. Two existence-preserving directions suffice for the no-solution theorem, provided both domains are retained.

Use the smallest useful helper. The same-domain 69/70/71 equivalences in `CurrentCarrierArithmetic2011.lean:95–121` preserve the thirteen-coordinate assignment; `hasSolution_congr` may remove repetitive packaging there. The 65-to-13 completeness map instead constructs fresh private Pell seeds, and the reverse restores a coordinate using an absolute value of a signed ordinate. These prove existence equivalence and preserved outer data, not automatically a bijection. Do not build a hierarchy merely to shorten two proofs: an existing short `Iff.trans` chain is already good.

The guards are mathematical content. In the nine-coordinate natural shift, coordinate 7 is left unchanged and the other eight receive $+1$. The shared-tail prototype retains that exact `successorShift`; it must not become an all-positive shift. Pell lifts need nonnegativity/positivity, and product-of-unit proofs need their sign exclusions.

Regression obligations.

- Zero remains permitted at the unshifted coordinate where the original theorem permits it.
- Strict positivity holds at the other coordinates and all inserted witnesses.
- Existential quantifier order and parameter dependence of witness choices remain unchanged.
- Integer equations and sign conventions remain exact, not merely valid on an inadvertently smaller domain.

Use the transport interface only at repeated boundaries; keep arithmetic witness-map proofs local. It does not supply a missing current-polynomial/native-machine map or an internal-ZFC derivation. Those are mathematical obligations, not packaging that an abstract structure can discharge.

11 Preserve DAG cutpoints while reducing duplication

Two generated files dominate a conspicuous part of the source: `SplitCircuitSteps59.lean` has 3,445 lines, 515 one-node facts and 107 power facts; `GapCircuitSteps66.lean` has 3,122 lines, 484 one-node facts and 86 power facts. Together they contain **6,567 lines**, **999 node lemmas** and **193 power lemmas**. That is 8.88% of main Lean source lines and 25.41% of lexical theorem/lemma declarations, not a forecast of removable work.

Do not delete these facts because their proofs are one-liners. Node lemmas specialize `ArithmeticDAGSemantics.run_node_equation`; power lemmas compress chains before clients unfold them. Register and recipe proofs use selected `simp only` lists instead of reducing a whole circuit. These named cuts may reduce elaboration or kernel-reduction costs. A shorter generated file can be a slower proof.

Three increasingly invasive options.

1. **Share generation templates.** Consolidate Split/Gap generation logic and optionally emit a local step helper hiding repeated topology/length arguments. Preserve public theorem names and intermediate cuts. This mainly improves maintenance.
2. **Factor common recipe-tail correctness.** Both recipe files expand the identical tail into 22 lets and establish register equalities. Keep concrete indices generated; prove the common consequence from input-register equalities and local gate equations. Their `extract_lets`/`clear_value` discipline is evidence that sharing matters.
3. **Use a checked finite semantic certificate.** Generate local-equation data, validate it with a proved checker, and recover the run via `run_eq_of_node_equations`. Retain named cuts used downstream. This is an architectural experiment, not a safe search-and-replace.

Degree accounting is a smaller companion change. `DAGQuartic.lean:214–234` and `NaturalDAGQuartic.lean:222–242` repeat an RHS degree case split. A generic node-expression constructor can assume every wire expression has degree at most one and conclude every RHS has degree at most two. Start by sharing the smaller addition/subtraction degree helpers. Keep signed wires, natural wires, nonnegativity assumptions and variable counts separate.

A structure is not automatically a simplification. A tail-register record with twenty-plus fields can merely move verbosity. Compare its concrete instance plus consumer proof with the original. If projections require extensive unfolding or prevent sharing, prefer generator-template reuse, which can remove duplicated maintenance without redesigning Lean terms.

Acceptance. Preserve node order, topology, output indices, counted operation lists, coefficients, degrees and variable counts. Benchmark generated output and representative coefficient/recipe clients. Do not replace the certificate story with `native_decide`, one enormous `ring`, or an unproved external check. The goal is simpler maintenance with acceptable checking cost, not fewer generated lines at any price.

12 Native proofs need smaller contexts and local equations

Project a small context from existing facts. `CurrentPolynomialWindow2011.lean:27–63` already packages broad provenance in `Facts`. Downstream count and decode proofs repeatedly pass the same five ingredients: Boolean digits, support, carry bound, exact carry equation and high-mask tests. The proposed `WordFacts` is a small `Prop` record containing precisely these, with an adapter from `Facts`. It is not the first record abstraction in the project.

Targets include `CurrentOneHot2011.lean:44–98`, `CurrentCellLabels2011.lean:214–282`, and `CurrentDecodedWindow2011.lean:23–70`. The prototype makes calls of the shape

```
h.count_le_one cell
h.decode_cell hphase cell
```

possible without five parallel argument lists. Crucially, the initial context does *not* assume phase zero, origin or occupation. Extra stages must derive their new fields:

$$\text{normalized word} \longrightarrow \text{count and phase} \longrightarrow \text{decoded word}.$$

Whole alignment and occupied tilings are later obligations, not fields to assume in order to shorten current proofs. A single large record can conceal circular dependencies; prefer small explicit conversions and migrate two consumers first.

Add equation-side carry lemmas, not a second carry theory. The generic recurrence/fixed-point API already exists in `NativeCyclicCarry.lean:23–105`. Several consumers instead start from

$$T = d + Bq \qquad (T, d, B, q \in \mathbb{N}).$$

Two reusable consequences are:

$$T < B \Longrightarrow (q = 0 \wedge d = T), \qquad B \bmod m = 0 \Longrightarrow T \bmod m = d \bmod m.$$

The first needs no separate digit bound: the small total forces zero outgoing carry. The second does not assume carry zero. `NativeSmallLemmas.lean` gives both candidates. Apply them after the appropriate raw budget, for example at `CurrentEarlyPadding2011.lean:80–80` and `CurrentEarlyPadding2011.lean:237–237`. Keep the early budget used to prove one-hot separate from later budgets that assume it.

Reuse the existing aligned-cell type. `CurrentCellTranslation2011.lean:19–54` already has `Cell` and its translation equivalence. Wrappers can centralize residue/nonzero facts for an offset $0 < e < 2011$, avoiding repeated arithmetic in padding and marker proofs. Preserve the global-origin exception: high-mask digit 3 at zero differs from normal aligned-cell digit 4. Also retain the $N = 1$ and full-period translation checks. If earlier modules cannot import the existing cell module without a cycle, extract a small shared geometry core rather than inventing a competing subtype.

13 Finite dispatch and literal native data

Keep kernel-checked chunks; simplify their assembly. `WangTiles942Data.lean:210–320` dispatches 27 action chunks covering 430 entries; the final block starts at 416 and has length 14. `WangTiles942Data.lean:439–513` dispatches 14 rail chunks covering 216 entries per direction; the last starts at 208 and has length 8. The selected dispatch regions have 111 and 75 physical lines respectively.

The chunks bound temporary normalization memory. Replacing them by one large `decide` is not demonstrably better. Instead prove adjacent-block coverage once, then generate a balanced composition of existing chunk theorems. `CertificateBlocks.lean` supplies the proposed `HoldsBlock.append` boundary. Keep per-block `decide +kernel` equalities to the original decoder. A generator emits ordinary proofs; it is not trusted. Coverage must include the final partial block.

Describe the packed direction layout once. Four control blocks repeat in `CurrentKernel2011.lean:43–57` and `CurrentCellLabels2011.lean:59–65`:

Direction	Width	Source start	Packed bit	Weight start
West	8	978	0	10
North	9	960	8	18
East	8	969	17	27
South	9	942	25	35

A finite direction descriptor plus a verified reindexing of the 34 controls could remove nested sum plumbing. It must preserve printed tile order, 942 selectors and three unit terms, and *sum* collisions rather than silently deduplicate them. The existing structural `recoveredIndex`/`strictRank` proofs are already good; do not replace them with an expensive sorting cache.

Optional mask-block reuse. The proof in `CurrentHighDigits2011.lean:45–107` already works symbolically over ten blocks. Extract a block lookup/length/bound library only if a second substantial mask client justifies it. Preserve three different operations: low-mask origin +1; repeated high-mask origin −1 (4 to 3); and scalar high-port shift by `beta`. These are not interchangeable cellwise adjustments.

Negative controls are essential. A phase-10 raw clause with 512 selectors passes the entire clause mask without padding information; clause masking alone does not prove one-hot. Offset 19 admits the actual 511 self-loop, so its theorem cannot be strengthened to exclude all cycles. The original negative controls are at `NativePreliminaryCount2011Checks.lean:13–16` and `NativeFieldCycles2011Checks.lean:14–21`. Python replay checks them, exact block coverage and 9,020 small carry-equation instances, but is not a Lean proof.

Future finite-state certificates may help local controller invariants only after a proved binding to the actual decoder. Closure proves preservation from an initial state; exclusion of all cycles needs additional ranking/SCC reasoning. None of this closes whole-field alignment, occupation/adjacency, native-machine semantics, converse-controller witnesses or internal ZFC.

14 Share control flow without collapsing cost models

Direct and native graph enumerators share source-level control flow but use intentionally different combinators. `rawStepProgram` is text-identical at `DirectGraphEnumerator.lean:177–201` and `NativeGraphEnumerator.lean:67–91`; `boundedRunProgram` repeats at `DirectGraphEnumerator.lean:218–228` and `NativeGraphEnumerator.lean:108–118`.

Proposed boundary. Parameterize the builder over the small set of differing graph operations: composition, branching, pairing/unpairing, and state/formula graph constructors. An explicit structure is preferable to a global typeclass unless instance search has a demonstrated benefit. Instantiate once for direct combinators and once for native combinators.

The programs are not interchangeable. The native step has **1,742 witnesses**; the direct step has **6,924**, recorded together at `NativeGraphEnumerator.lean:128–133`. A shared template must retain these exact values. Aliasing native to direct would destroy the optimization even if semantic predicates remained equivalent.

Expose correctness and cost projections separately. Correctness can be shared where assumptions match; cost must evaluate with actual realizers. Avoid a generic record whose instances become more verbose than the repeated builders. Start with the common terms and include adapters in the size comparison.

Imports are a separate engineering opportunity. The main umbrella has 283 direct imports. Narrow imports make experiments easier to isolate, and thematic umbrellas can improve navigation. But import organization alone is not a shorter proof, and accidental transitive dependencies should not replace deliberate imports.

An umbrella build does not cover all audits. The static local import closure reaches 325 of 364 primary modules, counting the umbrella itself. The 39 outside it are 38 modules named Checks/Audit plus `OnePageDisplay54`; the separate `checks/` directory has another 182 files outside that closure. This is an import-coverage finding, not a claim about which tests upstream has run. Maintain an explicit affected-check manifest in addition to the umbrella target.

Suggested module discipline.

- Put DivMod arithmetic below encoding-specific modules, and the common scalar tail below Split/Gap specializations.
- Put iteration mechanics below scanner invariants, using a small bridge to the existing library.
- Retain old import paths and theorem names through compatibility wrappers until the full consumer search is complete.
- Keep audit targets separate from production dependencies but run them when their boundary changes.

During migration report core-source reduction, compatibility overhead and total maintained source separately. A favorable local change can otherwise conceal a net increase.

15 What the concrete examples actually measure

Replacement proof	Before	After	Change
`Carrier66.drop_pos`	16	4	−12
`Carrier66.restore_pos`	17	5	−12
`Carrier66.canonical_zero`	4	2	−2
`QuarticCompiler.Gates.complete`	19	11	−8
`ShortSystem.residualSum_zero_iff`	23	2	−21
`ShortSystem.product_eq_one_iff`	17	13	−4
`ArithmeticDAG.run_map`	7	5	−2
`FirstPellCarrier.first_norm_facts_int`	16	13	−3
`ShortSystem.unit_eq_one`	4	1	−3
Nonblank proof-body lines	**123**	**56**	−67

Interpretation. This is 54.5% fewer nonblank body lines and 47.6% fewer body characters for nine selected examples, not a random sample of the project. Do not extrapolate to 73,971 lines. Declaration headers, comments, imports and shared-helper costs are excluded. The CSV and script preserve before/after text and counting conventions. The separate six-file library-cleanup patch has a net 70 physical lines removed; do not combine these different metrics into a purported verified total.

Static evidence established.

- All nine algebra declaration headers match the originals exactly.
- Their replacement text contains no new placeholder proof or axiom declaration.
- Five algebra patches and the standard-library cleanup patch pass textual applicability checks against the pinned tree.
- Relevant core/mathlib names were inspected in pinned source.
- Duplicate definitions and blocks were compared directly; finite Python sanity checks were replayed separately.

An unchanged header can still have an ill-typed body. An applicable patch can fail to elaborate. A token scan is not an axiom audit, and a library theorem's existence does not establish argument inference at a proposed use site.

Compilation status. All new Lean replacements and prototypes in this report remain **uncompiled**. No build-time, memory, proof-term-size or kernel-checking improvement is claimed. Existing upstream checked results are not being denied; they do not validate new replacement bodies.

Record next. For each accepted patch, retain the commit/toolchain, changed modules, fresh versus reused objects, command exit statuses, axiom output, total source accounting and representative elapsed-time/peak-memory measurements. Compare the same endpoint and dependencies. A successful helper check, an extracted-example check and a production build are distinct scopes; none should be described as a complete project validation unless it actually is one.

16 Acceptance gates and implementation order

Gate 1 Establish a baseline. Build the pinned source and record Lean/mathlib revisions. Choose representative endpoints: a small checker, an arithmetic carrier, a circuit/recipe client, and each affected native boundary. Record fresh and warm builds separately. Verify quantitative theorems and axiom reports before changing code.

Gate 2 Check low-risk candidates individually. Start with core list reuse and the DivMod certificate, then local algebra proofs. Compile changed modules before their reverse-dependency closure. Keep public names, arguments and conclusions. Narrow prototype imports only after successful elaboration, then rebuild.

Gate 3 Introduce small shared interfaces. Check the iteration bridge before replacing scanner wrappers, and exact output specifications before simplifying `step_cons`. Extract the scalar tail with old names retained and check intended definitional equalities. Keep assumptions readable beside a real consumer.

Gate 4 Protect every semantic boundary.

- Naturals versus integers, strict versus weak positivity, and all sign exclusions.
- Eight shifted coordinates versus one unshifted coordinate; witness dimensions and quantifier order.
- Raw encodings, supplied and insufficient fuel, zero and completed states.
- Polynomial equality, not finite evaluations; coefficient data and exponents.
- Signed/natural realizers; exact witness counts, variable counts, degrees and operations.
- Original premises and axiom sets; no trust-changing automation.

Gate 5 Measure structural experiments. Try one native context or finite-dispatch improvement before redesigning all endpoints. Consolidate generator templates before replacing named lemmas by new certificate machinery. Compare maintained source including adapters and certificates. A checking-cost regression may justify a longer proof.

Gate 6 Build both clients and explicit audits. Build the affected closure, main umbrella, and the separate relevant Checks/Audit/checks targets. The umbrella alone misses tests identified in Section 14. Preserve logs and `#print axioms` output for exact declarations. A standard axiom set does not compensate for a theorem weakened by a new hidden assumption.

Recommended first patch. Reuse core list facts, extract DivMod arithmetic, and simplify the two positivity proofs. This tests the acceptance workflow without changing representations. Follow with units/squares, scanner mechanics and shared tails. Leave native and generated-certificate changes for measured pilots.

Stopping criterion. Adopt a change when it simplifies a real maintenance task, preserves required contracts, passes relevant builds and axiom checks, and has acceptable checking cost. There is no obligation to turn every repeated line into an abstraction. Keep the mathematical gaps visible: no interface should replace an unproved native equivalence by a field that simply assumes it.

17 Sources and reproducibility guide

Primary source. Every ProveIt link resolves to the reviewed revision:

`60e9c5d620c6c387d932876cfa2d9376f64ea56c`.

The source tree is `Computability/DiophantineConsistency`. Section links identify declarations rather than only the repository home page.

Pinned library evidence. The mathlib source revision inspected for iteration and algebra APIs is `81a5d257c8e410db227a6665ed08f64fea08e997`. It includes `Mathlib.Logic.Function.Iterate` and the ordered-ring/group-with-zero facts for sums and squares. API evidence files retain exact excerpts. Core lookup/Option facts are pinned to Lean `v4.32.0`.

Artifact family	Purpose
`report/report.tex` and PDF	Editable and rendered analysis.
`algebra_examples/`	Before/after Lean bodies, patches, candidate metadata, source hashes, CSV and static audit.
`checker_examples/`	Iteration, scanner, library-cleanup, dispatch, closure and exact-output candidates.
`proposals/`	Shared DivMod certificate, scalar tail and intended compatibility checks.
`SolutionTransport.lean`	Guarded congruence and composable existence transport.
`prototypes/native/`	Context, carry and finite-dispatch prototypes; source index and finite sanity checks.
Evidence and validation files	Counts, source pins, API evidence and explicit verification status.

Static replay. Run `python3 -B verify.py` from the archive root. This rechecks packaged source evidence, patch applicability and finite sanity tests; it does not run Lean. The separate `validation/check_core.py` requires the pinned external compiler and an external `--output-dir`, keeping generated validation files out of the sealed evidence bundle.

Reproduction discipline. Apply patches to the recorded commit or review every changed context at a newer revision. Keep experiments in a separate checkout. Compile in the pinned Lake context with the declared imports: isolated extracts can omit private declarations or local instances. The included baseline subtree is not the full runnable repository dependency closure. For generated files, modify the producer as well as its output, or regeneration will undo the change.

Bottom line. Useful simplifications are available without changing mathematics: replace duplicate arguments with existing library facts, centralize common arithmetic/iteration laws, and preserve deliberate cost and reduction boundaries. Larger opportunities include shared polynomial tails and staged native/circuit proofs, but their acceptance must rest on exact contracts and integration evidence, not brevity alone.